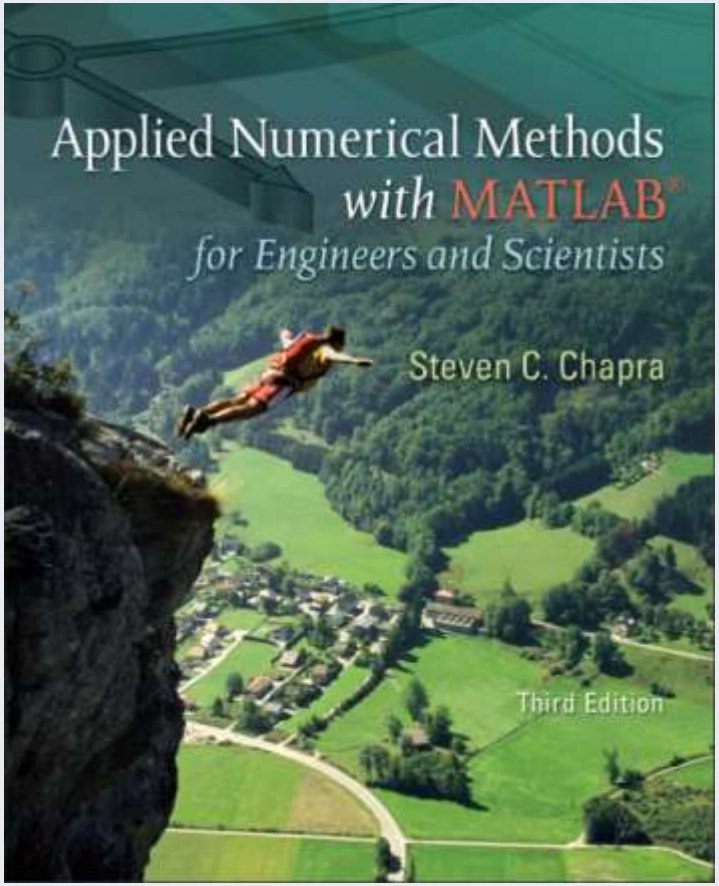


Part 4
Chapter 17

Polynomial Interpolation

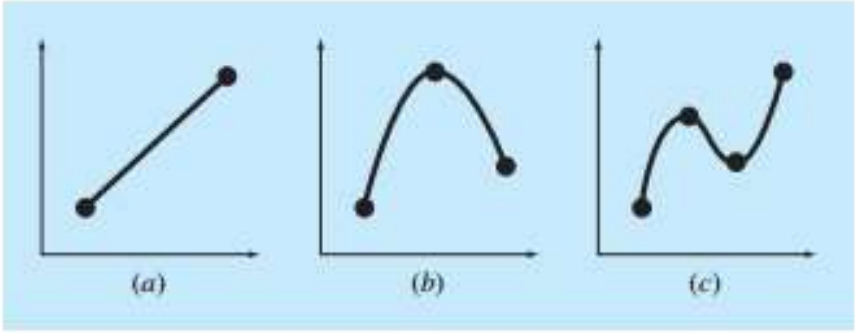
PowerPoints organized by Dr. Michael R. Gustafson II, Duke University

All images copyright © The McGraw-Hill Companies, Inc. Permission required for reproduction or display.

Chapter Objectives

- Recognizing that evaluating polynomial coefficients with simultaneous equations is an ill-conditioned problem.
- Knowing how to evaluate polynomial coefficients and interpolate with MATLAB's polyfit and polyval functions.
- Knowing how to perform an interpolation with Newton's polynomial.
- Knowing how to perform an interpolation with a Lagrange polynomial.
- Knowing how to solve an inverse interpolation problem by recasting it as a roots problem.
- Appreciating the dangers of extrapolation.
- Recognizing that higher-order polynomials can manifest large oscillations.

Examples of interpolating polynomials: (a) first-order (linear) connecting two points, (b) second-order (quadratic or parabolic) connecting three points, and (c) third-order (cubic) connecting four points.



Polynomial Interpolation

- You will frequently have occasions to estimate intermediate values between precise data points.
- The function you use to interpolate must pass through the actual data points - this makes interpolation more restrictive than fitting.
- The most common method for this purpose is polynomial interpolation, where an (n-1)th order polynomial is solved that passes through n data points:

$$f(x) = a_1 + a_2 x + a_3 x^2 + \cdots + a_n x^{n-1}$$

MATLAB version :

$$f(x) = p_1 x^{n-1} + p_2 x^{n-2} + \cdots + p_{n-1} x + p_n$$

Determining Coefficients

- Since polynomial interpolation provides as many basis functions as there are data points (*n*), the polynomial coefficients can be found exactly using linear algebra.
- MATLAB's built in polyfit and polyval commands can also be used - all that is required is making sure the order of the fit for *n* data points is *n*-1.

Polynomial Interpolation Problems

- One problem that can occur with solving for the coefficients of a polynomial is that the system to be inverted is in the form:

$$\begin{bmatrix} x_1^{n-1} & x_1^{n-2} & \cdots & x_1 & 1 \\ x_2^{n-1} & x_2^{n-2} & \cdots & x_2 & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{n-1}^{n-1} & x_{n-1}^{n-2} & \cdots & x_{n-1} & 1 \\ x_n^{n-1} & x_n^{n-2} & \cdots & x_n & 1 \end{bmatrix} \begin{Bmatrix} p_1 \\ p_2 \\ \vdots \\ p_{n-1} \\ p_n \end{Bmatrix} = \begin{Bmatrix} f(x_1) \\ f(x_2) \\ \vdots \\ f(x_{n-1}) \\ f(x_n) \end{Bmatrix}$$

- Matrices such as that on the left are known as *Vandermonde matrices*, and they are very ill-conditioned - meaning their solutions are very sensitive to round-off errors.
- The issue can be minimized by scaling and shifting the data.

Newton Interpolating Polynomials

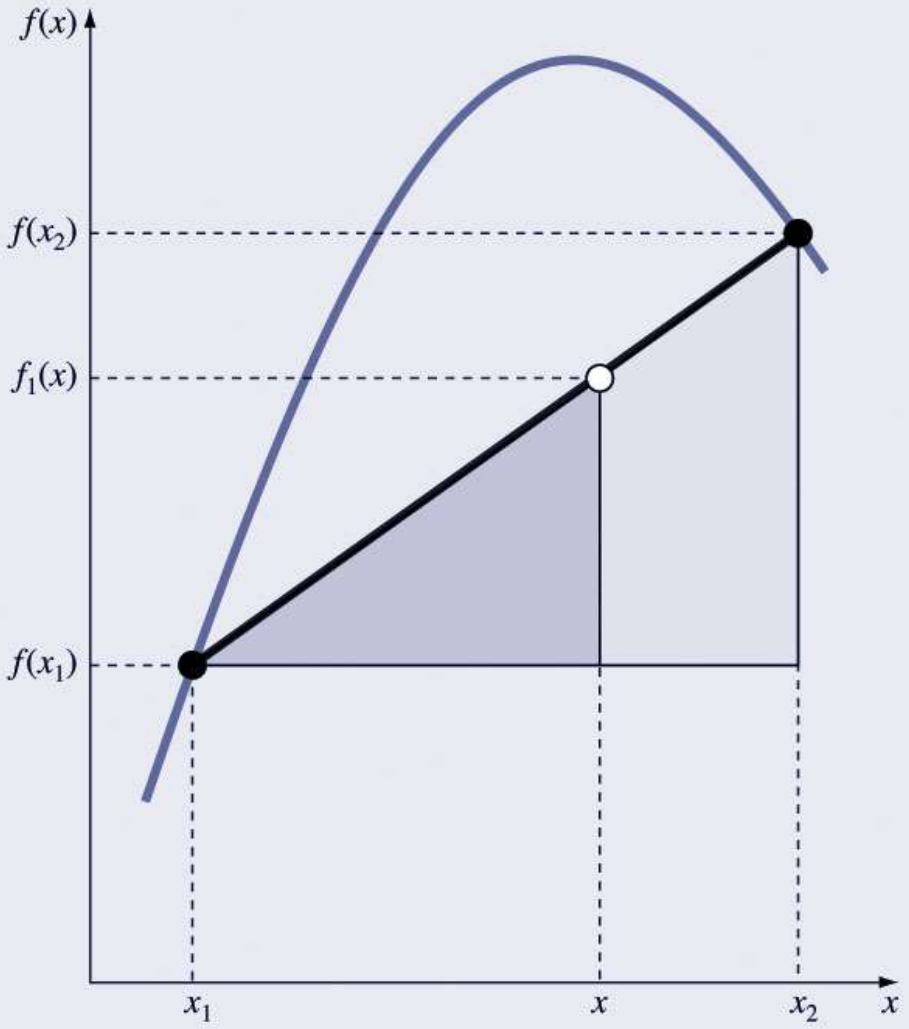
- Another way to express a polynomial interpolation is to use *Newton's interpolating polynomial*.
- The differences between a simple polynomial and Newton's interpolating polynomial for first and second order interpolations are:

Order	Simple	Newton
1st	$f_1(x) = a_1 + a_2 x$	$f_1(x) = b_1 + b_2(x - x_1)$
2nd	$f_2(x) = a_1 + a_2 x + a_3 x^2$	$f_2(x) = b_1 + b_2(x - x_1) + b3(x - x_1)(x - x_2)$

Newton Interpolating Polynomials (cont)

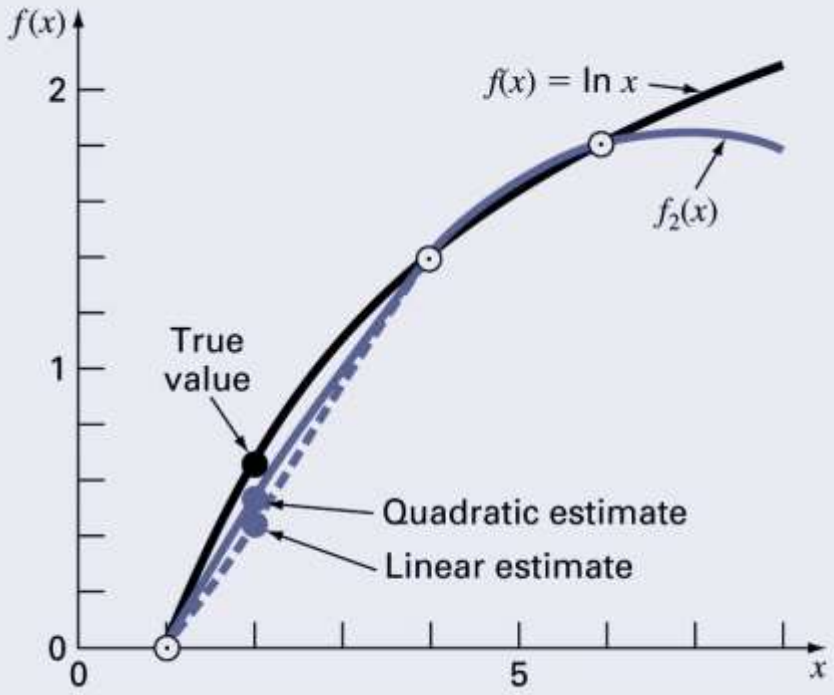
- The first-order Newton interpolating polynomial may be obtained from linear interpolation and similar triangles, as shown.
- The resulting formula based on known points x_1 and x_2 and the values of the dependent function at those points is:

$$f_1(x) = f(x_1) + \frac{f(x_2) - f(x_1)}{x_2 - x_1}(x - x_1)$$



Newton Interpolating Polynomials (cont)

- The second-order Newton interpolating polynomial introduces some curvature to the line connecting the points, but still goes through the first two points.
- The resulting formula based on known points x_1, x_2, and x_3 and the values of the dependent function at those points is:



$$f_2(x) = f(x_1) + \frac{f(x_2) - f(x_1)}{x_2 - x_1}(x - x_1) + \frac{\frac{f(x_3) - f(x_2)}{x_3 - x_2} - \frac{f(x_2) - f(x_1)}{x_2 - x_1}}{x_3 - x_1}(x - x_1)(x - x_2)$$

Newton Interpolating Polynomials (cont)

The general formula is:

$$f_{n-1}(x) = b_1 + b_2(x - x_1) + \cdots + b_n(x - x_1)(x - x_2)\cdots(x - x_{n-1})$$

where

$$b_1 = f(x_1)$$
$$b_2 = f[x_2, x_1]$$
$$b_3 = f[x_3, x_2, x_1]$$
$$\vdots$$
$$b_n = f[x_n, x_{n-1}, \cdots, x_2, x_1]$$

and the *f*[…] represent *divided differences*.

Divided Differences

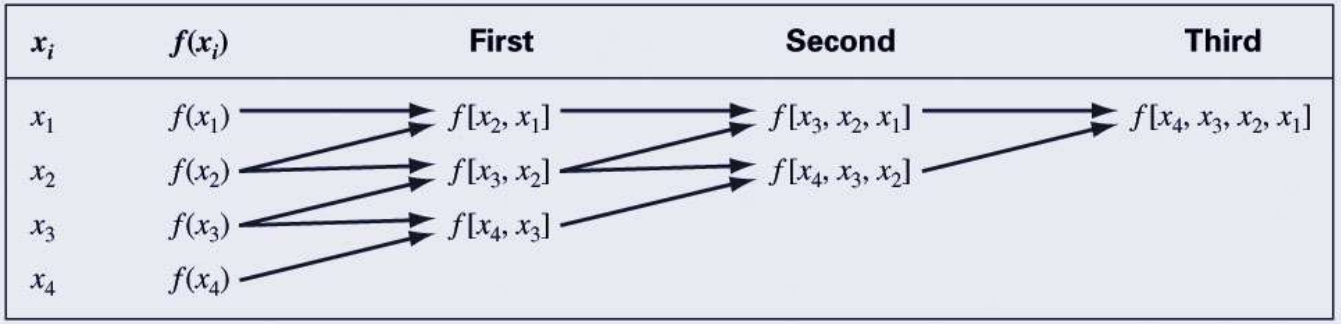
- Divided difference are calculated as follows:

$$f[x_i, x_j] = \frac{f(x_i) - f(x_j)}{x_i - x_j}$$

$$f[x_i, x_j, x_k] = \frac{f[x_i, x_j] - f[x_j, x_k]}{x_i - x_k}$$

$$f[x_n, x_{n-1}, \cdots, x_2, x_1] = \frac{f[x_n, x_{n-1}, \cdots, x_2] - f[x_{n-1}, x_{n-2}, \cdots, x_1]}{x_n - x_1}$$

- Divided differences are calculated using divided difference of a smaller number of terms:



MATLAB Implementation

```
function yint = Newtint(x,y,xx)
% Newtint: Newton interpolating polynomial
% yint = Newtint(x,y,xx): Uses an (n - 1)-order Newton
%   interpolating polynomial based on n data points (x, y)
%   to determine a value of the dependent variable (yint)
%   at a given value of the independent variable, xx.
% input:
%   x = independent variable
%   y = dependent variable
%   xx = value of independent variable at which
%        interpolation is calculated
% output:
%   yint = interpolated value of dependent variable

% compute the finite divided differences in the form of a
% difference table
n = length(x);
if length(y)~=n, error('x and y must be same length'); end
b = zeros(n,n);
% assign dependent variables to the first column of b.
b(:,1) = y(:);  % the (:) ensures that y is a column vector.
for j = 2:n
  for i = 1:n-j+1
    b(i,j) = (b(i+1,j-1)-b(i,j-1))/(x(i+j-1)-x(i));
  end
end
% use the finite divided differences to interpolate
xt = 1;
yint = b(1,1);
for j = 1:n-1
  xt = xt*(xx-x(j));
  yint = yint+b(1,j+1)*xt;
end
```

Lagrange Interpolating Polynomials

- Another method that uses shifted value to express an interpolating polynomial is the *Lagrange interpolating polynomial*.
- The differences between a simply polynomial and Lagrange interpolating polynomials for first and second order polynomials is:

Order	Simple	Lagrange
1st	$f_1(x) = a_1 + a_2 x$	$f_1(x) = L_1 f(x_1) + L_2 f(x_2)$
2nd	$f_2(x) = a_1 + a_2 x + a_3 x^2$	$f_2(x) = L_1 f(x_1) + L_2 f(x_2) + L_3 f(x_3)$

where the L_i are weighting coefficients that are functions of x.

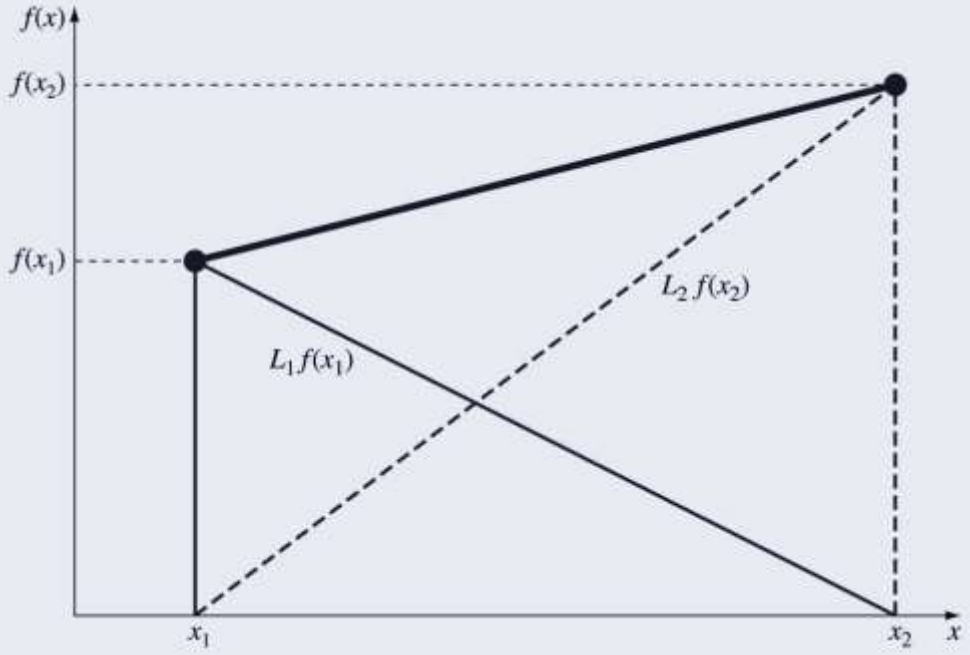
Lagrange Interpolating Polynomials (cont)

- The first-order Lagrange interpolating polynomial may be obtained from a weighted combination of two linear interpolations, as shown.
- The resulting formula based on known points x_1 and x_2 and the values of the dependent function at those points is:

$$f_1(x) = L_1 f(x_1) + L_2 f(x_2)$$

$$L_1 = \frac{x - x_2}{x_1 - x_2}, L_2 = \frac{x - x_1}{x_2 - x_1}$$

$$f_1(x) = \frac{x - x_2}{x_1 - x_2} f(x_1) + \frac{x - x_1}{x_2 - x_1} f(x_2)$$



$$f(x) = f(x_1) + \frac{f(x_2) - f(x_1)}{x_2 - x_1}(x - x_1)$$

Lagrange Interpolating Polynomials (cont)

- In general, the Lagrange polynomial interpolation for *n* points is:

$$f_{n-1}(x_i) = \sum_{i=1}^{n} L_i(x) f(x_i)$$

where L_i is given by:

$$L_i(x) = \prod_{\substack{j=1 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}$$

MATLAB Implementation

```
function yint = Lagrange(x,y,xx)
% Lagrange: Lagrange interpolating polynomial
%   yint = Lagrange(x,y,xx): Uses an (n - 1)-order
%     Lagrange interpolating polynomial based on n data points
%     to determine a value of the dependent variable (yint) at
%     a given value of the independent variable, xx.
% input:
%   x = independent variable
%   y = dependent variable
%   xx = value of independent variable at which the
%        interpolation is calculated
% output:
%   yint = interpolated value of dependent variable

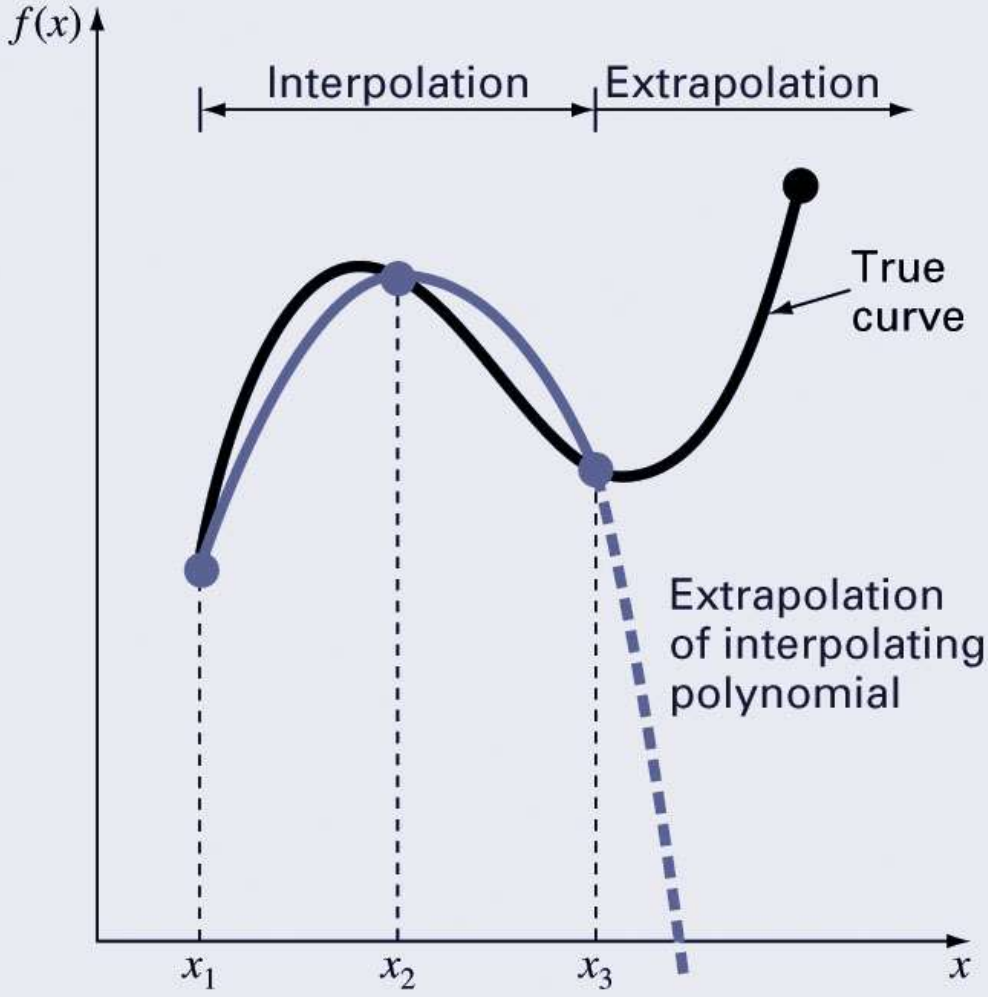
n = length(x);
if length(y)~=n, error('x and y must be same length'); end
s = 0;
for i = 1:n
  product = y(i);
  for j = 1:n
    if i ~= j
      product = product*(xx-x(j))/(x(i)-x(j));
    end
  end
  s = s+product;
end
yint = s;
```

Inverse Interpolation

- Interpolation general means finding some value $f(x)$ for some x that is between given independent data points.
- Sometimes, it will be useful to find the x for which $f(x)$ is a certain value - this is *inverse interpolation*.
- Rather than finding an interpolation of x as a function of $f(x)$, it may be useful to find an equation for $f(x)$ as a function of x using interpolation and then solve the corresponding roots problem: $f(x)-f_{desired}=0$ for x.

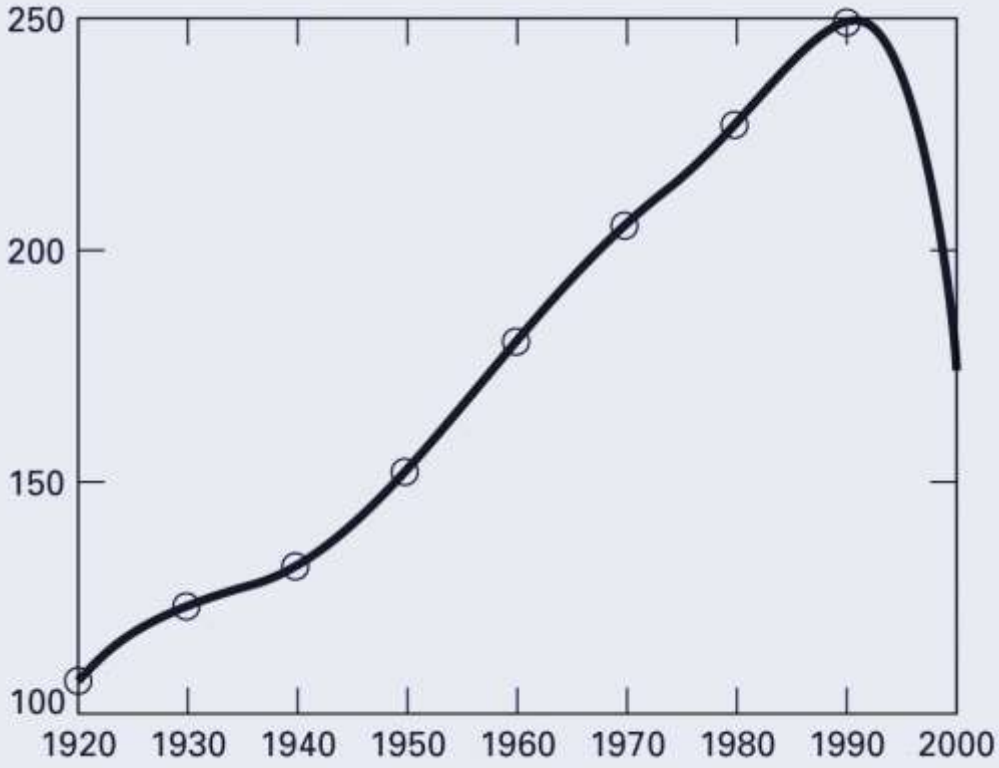
Extrapolation

- *Extrapolation* is the process of estimating a value of $f(x)$ that lies outside the range of the known base points x_1, x_2, …, x_n.
- Extrapolation represents a step into the unknown, and extreme care should be exercised when extrapolating!



Extrapolation Hazards

- The following shows the results of extrapolating a seventh-order population data set:



Oscillations

- Higher-order polynomials can not only lead to round-off errors due to ill-conditioning, but can also introduce oscillations to an interpolation or fit where they should not be.
- In the figures below, the dashed line represents an function, the circles represent samples of the function, and the solid line represents the results of a polynomial interpolation:

